The Speedrun Leaderboard

A running example, built one checkpoint at a time

Speedrunners compete to finish a video game as fast as possible, and a leaderboard records each run: the player, the game and category, the time, the date, and whether moderators verified it. In this guide we build a program that reads submitted runs from a file, rejects the bad ones, and reports world records, personal bests, and rankings.

We build it in class, one checkpoint at a time, from Week 12 to Week 14, and this guide contains every step, including any we do not reach. Each checkpoint prepares you for part of Lab W14, which repeats the same process with weather data but different details, so you need to understand each step, not copy it.

To see a real leaderboard, visit `https://www.speedrun.com/sm64`, which includes the 16 Star and 120 Star categories used in our data. Our game titles and categories are real, but the players and times are invented.

A note about this guide. I developed this guide with help from AI, which drafted much of the code, data, and explanations, and I review and revise them. Every program was compiled and run, and every output shown is real. If something looks wrong, does not match what you see in Eclipse, or contradicts the book or the notes, tell me. This guide shows how to build one program. It does not replace the notes, the assigned chapters, or the recorded lectures.

1. How to Use This Guide

Each checkpoint is a complete Eclipse project that compiles and runs. Every chapter of this guide starts from one project and ends at the next, as Table 1 shows. That means you can start any chapter, even if you missed the class before it: import the project the chapter starts from, and work through the steps.

Table 1: The Eclipse projects, and where each one is used.

Project	Use it to
`SpeedrunStarter`	Start the Starter Program chapter and Checkpoint 1
`SpeedrunCheckpoint1`	Check your Checkpoint 1 work, or start Checkpoint 2
`SpeedrunCheckpoint2`	Check your Checkpoint 2 work, or start Checkpoint 3
`SpeedrunCheckpoint3`	Check your Checkpoint 3 work, or start Checkpoint 4
`SpeedrunCheckpoint4`	Check your Checkpoint 4 work, or start Checkpoint 5
`SpeedrunCheckpoint5`	Check your Checkpoint 5 work, or start Checkpoint 6
`SpeedrunCheckpoint6`	Check your Checkpoint 6 work: the finished program

Importing a project into Eclipse

1. Download the project's zip file. You do not need to unzip it.
2. In Eclipse, choose **File**, then **Import**. Open **General**, choose **Existing Projects into Workspace**, and click **Next**.
3. Choose **Select archive file**, browse to the zip file, make sure the project is checked, and click **Finish**.

Every project has the same layout: a `speedrun` package for the classes that represent the data, an `app` package for the program with `main`, and a `files` folder for the data files.

How each chapter works

Each chapter lists the project it starts from, the project it ends at, and the Lab W14 tasks it prepares you for. Then it gives numbered steps. Each step shows the code to type and the output you should see when you run it. If your output does not match, stop and compare your code with the listing before going on, because each step builds on the one before it. Each chapter ends with *Check yourself* questions and their answers.

2. The Data

The file `files/runs.csv` holds one submitted run per line, after a header line. The runs are in the order they were submitted, not in any useful order:

```
player,game,category,time,date,verified
NovaByte,Super Mario 64,16 Star,16:51.5,2025-09-03,no
Kestrel,The Legend of Zelda: Ocarina of Time,100%,4:05:54.7,2025-11-11,no
```

Times are written the way speedrunners write them: minutes, seconds, and tenths, such as `16:51.5`, or hours first when a run takes an hour or more, such as `4:05:54.7`.

Not every line is good. Table 2 lists what the leaderboard accepts. A line that does not have exactly six fields, or has a field the table does not accept, is a *bad line*. The file `files/runs_clean.csv` has eight good runs and no bad lines, for testing.

Table 2: What the leaderboard accepts in each field.

Field	Accepted values
`player`, `game`, `category`	Text that is not blank
`time`	A time written as `M:SS.s` or `H:MM:SS.s`, with minutes and seconds below 60, and greater than 0
`date`	A real date written as year, month, and day, such as `2026-02-14`, and not in the future
`verified`	Exactly `yes` or `no`

3. The Starter Program

Starts from: SpeedrunStarter. **Changes nothing:** this chapter only runs it.

The starter program reads the clean file, splits each line on commas, turns the time and the date into Java objects, and prints each run. Here is all of it:

```
public class SpeedrunReport {

    public static void main(String[] args) throws IOException {
        List<String> lines = Files.readAllLines(Path.of("files/runs_clean.csv"));

        // Line 0 is the header, so the runs start at line 1.
        for (int i = 1; i < lines.size(); i++) {
            String[] fields = lines.get(i).split(",");
            String player = fields[0];
            String game = fields[1];
            String category = fields[2];
            Duration time = RunTime.parse(fields[3]);
            LocalDate date = LocalDate.parse(fields[4]);
            boolean verified = fields[5].equals("yes");

            System.out.printf("%-10s %-36s %-8s %10s  %s  %s%n",
                    player, game, category, RunTime.format(time), date,
                    verified ? "verified" : "unverified");
        }
    }
}
```

Three things in it may be new:

- `Files.readAllLines` reads the whole file into a `List<String>`, one element per line.
- `Duration` is Java's class for an amount of time, in the `java.time` package, next to `LocalDate`.
- `RunTime` is a small class in the `speedrun` package that is provided for you. `RunTime.parse` turns text such as `16:51.5` into a `Duration`, and `RunTime.format` turns a `Duration` back into text. Use it as it is.

Step 1: Run it on the clean file

Import SpeedrunStarter, open `SpeedrunReport.java` in the `app` package, and run it.

```
NovaByte   Super Mario 64                      16 Star     16:51.5  2025-09-03  unverified
Tangent    Super Mario 64                      16 Star     15:42.6  2025-09-24  unverified
Kestrel    The Legend of Zelda: Ocarina of Time 100%      4:05:54.7  2025-11-11  unverified
Lumen      Super Mario 64                      16 Star     17:03.4  2025-11-15  verified
Lumen      Super Mario 64                      16 Star     17:06.4  2025-12-13  verified
NovaByte   Super Mario 64                      120 Star  1:43:33.5  2025-12-19  verified
Moss       The Legend of Zelda: Ocarina of Time 100%      3:50:02.6  2025-12-25  unverified
Sprocket   Super Mario 64                      120 Star  1:38:42.1  2025-12-28  verified
```

Step 2: Run it on the real file

Change `runs_clean.csv` to `runs.csv` in the file name, and run it again.

```
NovaByte   Super Mario 64                        16 Star     16:51.5  2025-09-03  unverified
Tangent    Super Mario 64                        16 Star     15:42.6  2025-09-24  unverified
Kestrel    The Legend of Zelda: Ocarina of Time 100%      4:05:54.7  2025-11-11  unverified
Lumen      Super Mario 64                        16 Star     17:03.4  2025-11-15  verified
Lumen      Super Mario 64                        16 Star     17:06.4  2025-12-13  verified
Exception in thread "main" java.lang.NumberFormatException: For input string: "4x.1"
        at java.base/jdk.internal.math.FloatingDecimal.readJavaFormatString(FloatingDecimal.java
    :2054)
        at java.base/jdk.internal.math.FloatingDecimal.parseDouble(FloatingDecimal.java:110)
        at java.base/java.lang.Double.parseDouble(Double.java:792)
        at speedrun.RunTime.parse(RunTime.java:38)
        at app.SpeedrunReport.main(SpeedrunReport.java:30)
```

Read a stack trace from the top. The first line says *what* went wrong: a `NumberFormatException`, because `"4x.1"` is not a number. The lines below it say *where*, starting with the method that threw it and ending with `main`. Skip the lines inside `java.base`, which are Java's own code, and look for the first line in code from this project. Here, `RunTime.parse` threw the exception while `main` was calling it on line 30.

The bad line is line 7 of the file:

```
Kestrel,Celeste,Any%,28:4x.1,2025-10-02,yes
```

Someone typed an `x` instead of a digit. The real problem is what happened next: nothing. The program stopped, and the 36 good runs after that line were never read. One typo threw away the whole leaderboard.

Step 3: Run it with a missing file

Change the file name to `run_clean.csv`, without the `s`, and run it again.

```
Exception in thread "main" java.nio.file.NoSuchFileException: files/run_clean.csv
        at java.base/sun.nio.fs.UnixException.translateToIOException(UnixException.java:92)
        ... (ten more lines inside java.base)
        at app.SpeedrunReport.main(SpeedrunReport.java:22)
```

The real trace has ten more lines of Java's own code where the `...` is; your line numbers inside `java.base` may also differ, depending on your version of Java. The message itself is useful, because it names the file, but a person running the program should never see a stack trace. Change the file name back to `runs_clean.csv`.

4. Checkpoint 1: The File Problem and the Run Record

Starts from: SpeedrunStarter. **Ends at:** SpeedrunCheckpoint1. **Prepares you for:** Lab W14, Tasks 1 and 2.

This checkpoint fixes the missing-file problem from Step 3, and gives each run a record of its own that refuses bad values. It does not yet fix the crash from Step 2. That is Checkpoint 2.

Step 1: Why `throws` is not enough

The starter's `main` method is declared with `throws IOException`. A `throws` clause does not handle an exception. It passes the exception to whoever called the method. Nobody in your program calls `main`: Java does. So when an `IOException` leaves `main`, Java handles it the only way it can, by printing the stack trace and stopping. `main` is the top of the program, the last place your code can deal with the problem, so it must *catch* the exception instead.

Step 2: Catch the exception

Remove `throws IOException` from `main`, put the file name in a variable, and wrap the body of `main` in a `try` block with two `catch` blocks. Add `import java.nio.file.NoSuchFileException;` at the top of the file.

```
public static void main(String[] args) {
    String fileName = "files/runs_clean.csv";
    try {
        List<String> lines = Files.readAllLines(Path.of(fileName));

        // ... the for loop, exactly as before ...

    } catch (NoSuchFileException e) {
        System.out.println("Could not find the file " + e.getFile());
    } catch (IOException e) {
        System.out.println("Could not read the file " + fileName + ": " + e.getMessage
    ());
    }
}
```

- **Why two `catch` blocks?** A missing file is the most likely problem, and it deserves the clearest message. `NoSuchFileException` is a subclass of `IOException`, and its `getFile()` method returns the file's name. The second block catches every other `IOException`, such as a file you do not have permission to read.
- **Why this order?** Java tries the `catch` blocks from the top. If `IOException` came first, it would catch every `NoSuchFileException` too, and the second block could never run. The compiler refuses that order:

  ```
  error: exception NoSuchFileException has already been caught
  ```

  Always put the more specific exception first.
- **Why the `fileName` variable?** The second message needs the name, and an `IOException` does not always include it.

Step 3: Test it

Change the file name to `run_clean.csv` again, and run it:

```
Could not find the file files/run_clean.csv
```

One clear line, and no stack trace. Change the name back to `runs_clean.csv` and run it again. The output is the same as in the starter's Step 1.

Step 4: Write the Run record

Right now a run is six separate variables inside a loop. It should be one object. In the `speedrun` package, create a new *record* named `Run`. In Eclipse, choose **File**, **New**, **Record**. A record is declared with the keyword `record`, not `class`, and it is *immutable*: once a `Run` is created, its values can never change. So if the values are checked once, when the record is created, every `Run` that exists is good.

```
package speedrun;

import java.time.Duration;
import java.time.LocalDate;
import java.util.Objects;

public record Run(String player, String game, String category, Duration time,
        LocalDate date, boolean verified) {

    public Run {
        requireText(player, "player");
        requireText(game, "game");
        requireText(category, "category");
        Objects.requireNonNull(time, "time is required");
        Objects.requireNonNull(date, "date is required");
        if (time.isNegative() || time.isZero()) {
            throw new IllegalArgumentException(
                    "time must be greater than 0, but was " + RunTime.format(time));
        }
        if (date.isAfter(LocalDate.now())) {
            throw new IllegalArgumentException("date " + date + " is in the future");
        }
    }

    private static void requireText(String value, String name) {
        if (value == null || value.isBlank()) {
            throw new IllegalArgumentException(name + " is required");
        }
    }
}
```

Here is what each part does.

- **The header** lists the record's *components*. From that one line, Java writes the private final fields, a constructor that takes all six values, an accessor method for each component, and `equals`, `hashCode`, and `toString`. The accessors are named after the components, so you write `run.time()`, not `run.getTime()`.
- **The compact constructor**, `public Run { ... }`, has no parameter list. It runs before the values are stored, and Java stores them for you afterward, so you never write `this.time = time`. Its only job is to check the values.

- **A `throw` statement** stops the constructor. `new IllegalArgumentException(...)` creates the exception object, with a message that names the problem and includes the bad value. `throw` sends it to the caller immediately. The rest of the constructor does not run, and the `Run` is never created.
- **No `throws` clause is needed,** because `IllegalArgumentException` is unchecked. It means "the caller passed a bad value," and the compiler does not force anyone to plan for it.
- `Objects.requireNonNull` throws a `NullPointerException` with your message if the value is `null`, in one line.
- `requireText` is a private helper, so the three text checks are written once. A record may have its own methods, as long as they are not extra fields.
- `LocalDate.now()` is today's date, so a run dated after today is rejected.

Step 5: Try the record

Before using `Run` in the loop, try it on its own. Add these lines at the very top of `main`, and add `import speedrun.Run;` with the other imports:

```
Run good = new Run("Kestrel", "Celeste", "Any%", RunTime.parse("26:41.6"),
        LocalDate.of(2026, 2, 14), true);
System.out.println(good);
Run bad = new Run("Moss", "Portal", "Inbounds", RunTime.parse("0:00.0"),
        LocalDate.of(2025, 11, 14), true);
System.out.println(bad);
```

```
Run[player=Kestrel, game=Celeste, category=Any%, time=PT26M41.6S, date=2026-02-14, verified=true
    ]
Exception in thread "main" java.lang.IllegalArgumentException: time must be greater than 0, but
    was 0:00.0
        at speedrun.Run.<init>(Run.java:32)
        at app.SpeedrunReport.main(SpeedrunReport.java:27)
```

The good run prints with the `toString` that Java wrote. Notice how a `Duration` prints itself: `PT26M41.6S`, meaning a period of time of 26 minutes and 41.6 seconds. That is why the program uses `RunTime.format` instead. The bad run never exists: the constructor threw, and `<init>` in the trace is how Java names a constructor. Delete the six lines before going on.

Step 6: Use the record in the loop

Replace the six variables in the loop with one `Run`, and print it with its accessors:

```
for (int i = 1; i < lines.size(); i++) {
    String[] fields = lines.get(i).split(",");
    Run run = new Run(fields[0], fields[1], fields[2],
            RunTime.parse(fields[3]), LocalDate.parse(fields[4]),
            fields[5].equals("yes"));

    System.out.printf("%-10s %-36s %-8s %10s  %s  %s%n", run.player(),
            run.game(), run.category(), RunTime.format(run.time()),
```

```
            run.date(), run.verified() ? "verified" : "unverified");
}
```

Run it. The output is the same as in the starter's Step 1. Now change the file name to `runs.csv` and run it again. It still crashes on line 7, with the same `NumberFormatException` as before. Checkpoint 1 made bad *runs* impossible, but not bad *lines*: `RunTime.parse` fails before the `Run` constructor even runs, and if it had not, the constructor would throw instead. Either way, one exception still stops the whole loop. Change the file name back to `runs_clean.csv`.

Your project should now match SpeedrunCheckpoint1.

Check yourself

1. What does Java do with an exception that leaves `main`, and why is that a problem?
2. Why must the `catch` for `NoSuchFileException` come before the `catch` for `IOException`?
3. After this checkpoint, can a `Run` with a time of `0:00.0` exist anywhere in the program? Why or why not?
4. Line 31 of `runs.csv` has `maybe` in the verified field. What does the program do with it right now?

Answers

1. Java prints the stack trace and stops the program. The person running the program sees a wall of text meant for programmers, and nothing after the error runs.
2. `NoSuchFileException` is a subclass of `IOException`, so a `catch` for `IOException` placed first would catch it too, and the more specific block could never run. The compiler rejects that order.
3. No. The compact constructor throws before the record is created, and a record's values can never change afterward, so every `Run` that exists has a time greater than 0.
4. It quietly treats the run as unverified, because `fields[5].equals("yes")` is `false` for anything except `yes`. That is worse than a crash: the bad value is hidden. Checkpoint 2 makes it a bad line.